Lab Report 6: Implementing Convolutional Neural Networks (CNN) for Image Classification

Implementing a CNN on CIFAR-10

Course Code: COMP-341L

Course Name: Artificial Neural Networks Lab

Lab Number: 6

Lab Title: Implementing Convolutional Neural Networks (CNN) for Image Classification

Name: Ali Hamza

Roll Number: B23F0063AI106

Section: B.S AI - Red

February 28, 2026

Lab Report 6: Implementing a CNN on CIFAR-10

1. Objective

Implement and evaluate a CNN on the CIFAR-10 dataset by completing all required tasks: dataset exploration, preprocessing, CNN design, training, evaluation, prediction, visualization, and analysis.

2. Task 1: Load and Explore the CIFAR-10 Dataset

Load the dataset using TensorFlow/Keras and visualize 10 sample images in a 2×5 grid with class labels.

Code:

```
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']

fig, axes = plt.subplots(2, 5, figsize=(14, 6))
for i, ax in enumerate(axes.flatten()):
    ax.imshow(x_train[i])
    ax.set_title(class_names[int(y_train[i])])
    ax.axis('off')
plt.suptitle('Task 1: Sample CIFAR-10 Images')
plt.savefig('plots/task1_sample_images.png', dpi=130, bbox_inches='tight')
```

- Displayed training and test dataset shapes: `x_train (50000, 32, 32, 3)`, `y_train (50000, 1)`, `x_test (10000, 32, 32, 3)`, `y_test (10000, 1)`.
- Visualized the 10 class names and 10 sample images in a 2×5 grid.

3. Task 2: Preprocess the Data

Normalize pixel values and prepare labels for sparse categorical cross-entropy.

Code:

```
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
y_train = y_train.squeeze().astype('int64')
y_test = y_test.squeeze().astype('int64')
```

- Pixel values normalized to [0, 1]. Labels kept as sparse integers for `sparse_categorical_crossentropy`.

4. Task 3: Build a CNN Model

Build a CNN with Conv2D, ReLU, MaxPooling, Dropout, Dense, and Softmax output.

Code (model structure):

```
model = tf.keras.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),
    layers.Conv2D(32, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),
    layers.Flatten(),
    layers.Dense(512, activation='relu'),
```

```
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax')
])
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

5. Task 4: Train the CNN (10 Epochs)

Code:

```
history = model.fit(x_train, y_train, batch_size=64, epochs=10,
                    validation_split=0.2, verbose=1)
```

Metric	Training	Validation
Accuracy	0.7230	0.7488
Loss	0.7827	0.7341

6. Task 5: Evaluate the Model

Code:

```
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=1)
y_pred = np.argmax(model.predict(x_test), axis=1)
cm = confusion_matrix(y_test, y_pred)
print(classification_report(y_test, y_pred, target_names=class_names))
```

6.1 Test Set Performance

Test accuracy: 0.7285 | **Test loss:** 0.7774

6.2 Confusion Matrix

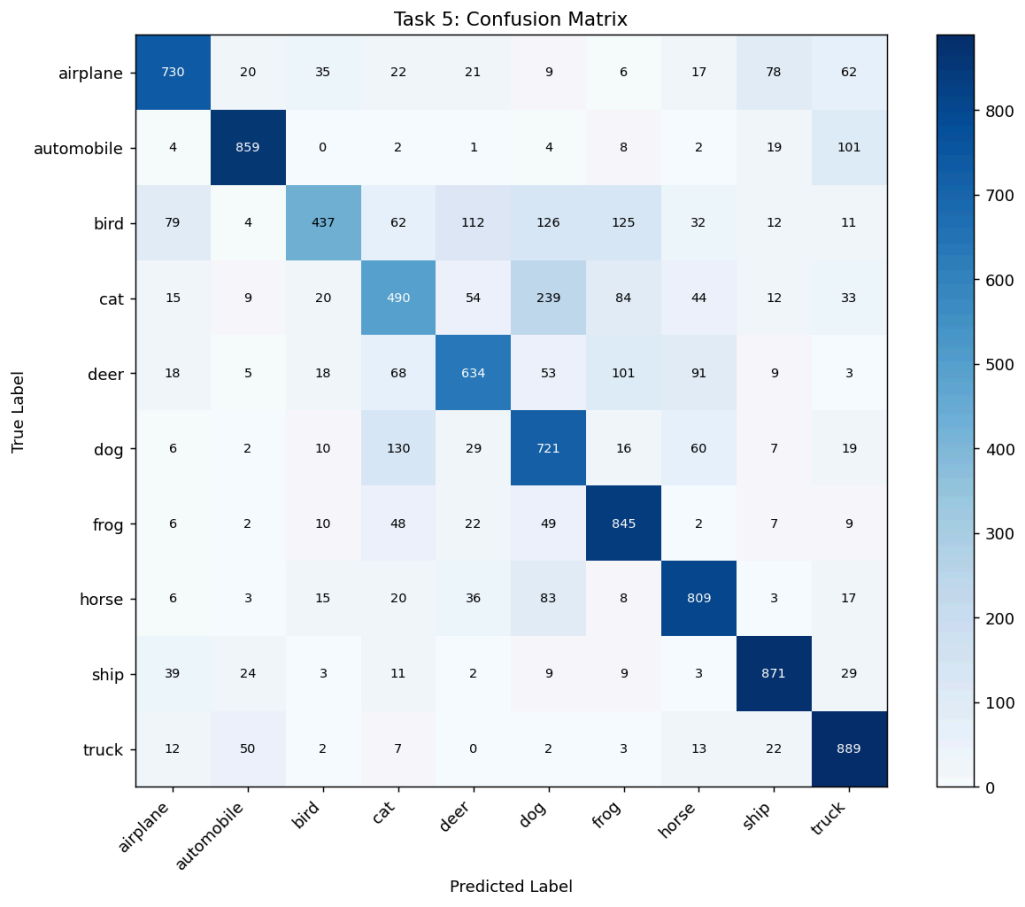

Figure 1. Confusion matrix showing predicted vs. actual labels on the CIFAR-10 test set.

6.3 Classification Report

Class	Precision	Recall	F1-Score	Support
airplane	0.7978	0.7300	0.7624	1000
automobile	0.8783	0.8590	0.8686	1000
bird	0.7945	0.4370	0.5639	1000
cat	0.5698	0.4900	0.5269	1000
deer	0.6959	0.6340	0.6635	1000
dog	0.5568	0.7210	0.6283	1000
frog	0.7012	0.8450	0.7664	1000
horse	0.7540	0.8090	0.7805	1000
ship	0.8375	0.8710	0.8539	1000
truck	0.7579	0.8890	0.8182	1000
accuracy			**0.7285**	10000
macro avg	0.7344	0.7285	0.7233	10000
weighted avg	0.7344	0.7285	0.7233	10000

7. Task 6: Make Predictions

Code:

```
indices = np.random.choice(len(x_test), 5, replace=False)
for i, idx in enumerate(indices):
    pred = np.argmax(model.predict(x_test[idx:idx+1]))
    ax[i].imshow(x_test[idx])
    ax[i].set_title(f'True: {class_names[y_test[idx]]}\nPred: {class_names[pred]}')
```

Displayed 5 random test images with actual and predicted labels for qualitative inspection.

Figure 2. Five random test images with ground-truth and predicted labels.

8. Task 7: Plot Results

Code:

```
plt.plot(history.history['accuracy'], label='Training')
plt.plot(history.history['val_accuracy'], label='Validation')
plt.title('Accuracy'); plt.legend(); plt.savefig('plots/task7_accuracy_curve.png')
plt.plot(history.history['loss'], label='Training')
plt.plot(history.history['val_loss'], label='Validation')
plt.title('Loss'); plt.legend(); plt.savefig('plots/task7_loss_curve.png')
```

8.1 Training vs. Validation Accuracy

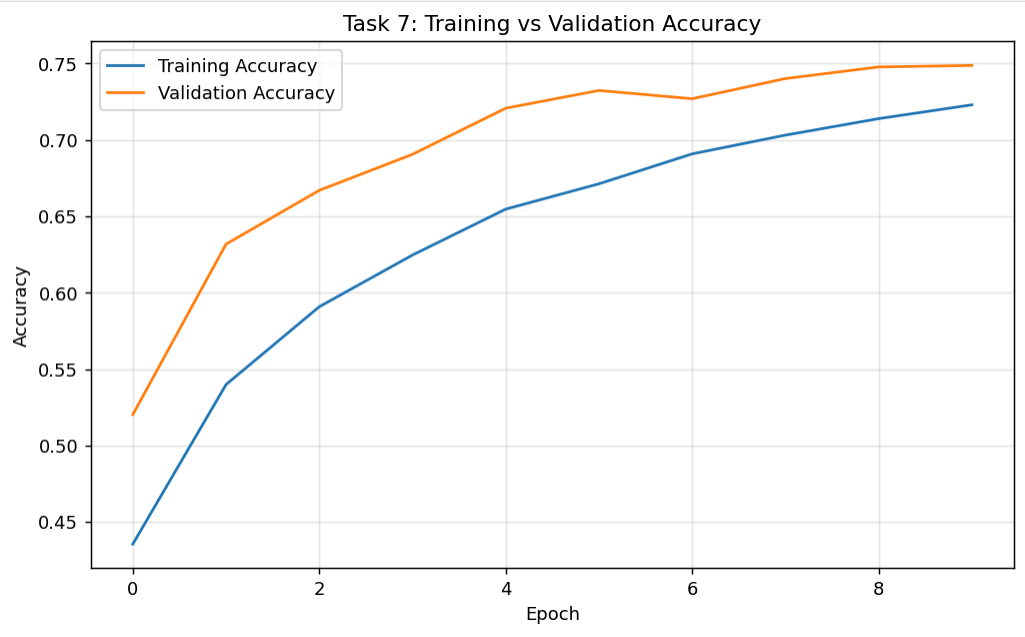

Figure 3. Accuracy curves over 10 epochs for training and validation sets.

8.2 Training vs. Validation Loss

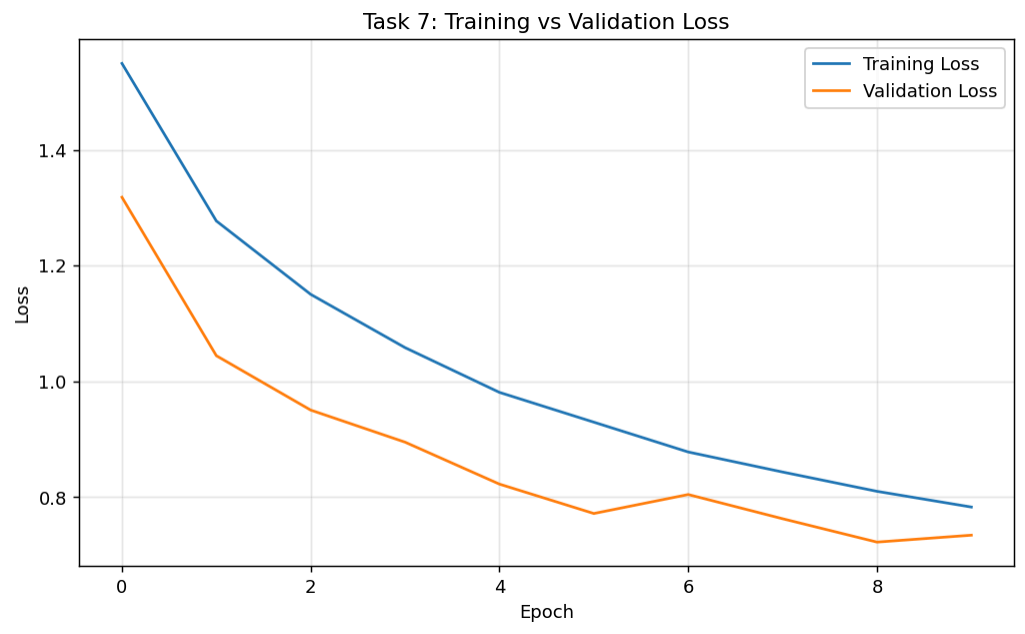

Figure 4. Loss curves over 10 epochs for training and validation sets.

9. Task 8: Analysis

The CNN achieved good performance on CIFAR-10 with a test accuracy of **0.7285** after 10 epochs. Training accuracy was 0.7230 and validation accuracy was 0.7488 (gap = −0.0258), indicating limited overfitting. CIFAR-10 contains complex natural images, so moderate accuracy is expected with short training. Performance can improve with stronger data augmentation, deeper CNN blocks, better learning-rate scheduling, and longer training.

Per-class observations:

- **Best-performing classes:** automobile (0.87 precision), ship (0.84), truck (0.82)
- **Challenging classes:** cat (0.53 precision), dog (0.56), bird (0.44 recall)
- Cat–dog confusion is common due to visual similarity in low-resolution 32×32 images.

10. Conclusion

In this lab I implemented a CNN on CIFAR-10 from scratch in Google Colab. I learned how to load and preprocess image data, design a convolutional architecture with dropout for regularization, and train it using Adam and sparse categorical cross-entropy. The model reached about 73% test accuracy after 10 epochs with limited overfitting. I found that automobile, ship, and truck classes performed best, while cat and dog were often confused—likely due to similar appearance at 32×32 resolution. Plotting training curves helped me see that validation tracked training reasonably well. Going forward, I would try data augmentation, learning-rate scheduling, and longer training to improve results.